Ingegneria del Software — Soluzione del Tema 15/02/2021

Esercizio 1

Si consideri la seguente classe Java `PackageManager` per gestire l'installazione di pacchetti software e delle loro dipendenze. I singoli pacchetti sono rappresentati dalla classe pura `Package` di cui riportiamo i metodi rilevanti.

```
public class PackageManager {
  // Ritorna lo spazio disco disponibile in KByte.
  public /*@ pure @*/ int availableSize();

  // Ritorna true se e solo se dep e' una dipendenza diretta o indiretta di p e
  // dep non e' attualmente installata.
  public /*@ pure @*/ boolean depToInstall(Package dep, Package p);

  // Se p non e' gia' installato e c'e' sufficiente spazio disco, installa p e tutti
  // i pacchetti da cui p dipende direttamente o indirettamente e che non sono installati.
  // Se non c'e' sufficiente spazio disco lancia l'eccezione DiskFullException.
  public void install(Package p) throws DiskFullException;

  // Ritorna l'elenco dei pacchetti attualmente installati
  public /*@ pure @*/ Set<Package> getInstalled();
}

public /*@ pure @*/ class Package {
  // Ritorna l'identificativo univoco del pacchetto software
  public String getId();

  // Ritorna la dimensione del pacchetto software in KBytes
  public int getSize();

  // Ritorna l'elenco dei pacchetti da cui questo pacchetto dipende direttamente
  public List<Package> getDeps();

  // Due pacchetti sono uguali se hanno lo stesso id
  public boolean equals(Object o);
}
```

Domanda a)

Si specifichi in JML il metodo `depToInstall()`.

Soluzione

```
//@requires dep != null && p != null
//@ensures \result <==> (
//@  ( p.equals(dep) && !getInstalled().contains(p) ) ||
//@  ( (\exists Package d; p.getDeps().contains(d); depToInstall(dep, d)) ) )
public /*@ pure @*/ boolean depToInstall(Package dep, Package p);
```

Domanda b)

Si specifichi in JML il metodo `install()`.

Soluzione

```
//@requires p != null
//@
//@ensures
//@ (\sum Package dep; \old(depToInstall(dep, p)); dep.size()) <= \old(availableSize()) &&
//@ (\forall Package x; ; getInstalled().contains(x) <==>
```

```
//@     ( \old(getInstalled()).contains(x) || depToInstall(x, p) ) ) &&
//@ availableSize() == \old(availableSize()) -
//@                    (\sum Package dep; depToInstall(dep, p); dep.size())
//@
//@signals DiskFullException &&
//@ (\sum Package dep; depToInstall(dep, p); dep.size()) > \old(availableSize()) &&
//@ getInstalled().containsAll(\old(getInstalled())) &&
//@ \old(getInstalled()).containsAll(getInstalled()) &&
//@ \old(availableSize()) == availableSize()
public  void install(Package p) throws DiskFullExcpetion;
```

Domanda c)

Si consideri un'implementazione che utilizza una lista per contenere i pacchetti installati e un intero per mantenere lo spazio disponibile, come mostrato di seguito.

```
public class PackageManager {
  private final List<Package> list;
  private int availableSize;
  ... }
```

Per tale implementazione, si definiscano l'invariante di rappresentazione e la corrispondente funzione di astrazione.

Soluzione

Nell'invariante di rappresentazione escludiamo che la lista sia nulla, che contenga valori nulli e che contenga duplicati. Richiediamo che lo spazio disponibile sia maggiore o uguale a zero

```
//@(* Representation invariant RI *)
//@private invariant list != null && availableSize >= 0 &&
//@ (\forall int i; i>=0 && i<list.size(); list.get(i) != null
//@   (\forall int j; j>=0 && j<i; !list.get(j).equals(list.get(i)) ) ) &&
```

La funzione di astrazione definisce `getInstalled()` in funzione della lista usata nella rappresentazione e `availableSize()` in funzione della variabile corrispondente. Gli altri metodi pubblici sono infatti definiti a partire da essi.

```
//@(* Abstraction function AF *)
//@private invariant availableSize() == availableSize &&
//@(\forall Package p; p!=null;
//@   getInstalled().contains(p) <==> list.contains(p) )
```

Domanda d)

Si consideri ora una classe `PackageManager2` che aggiunge un metodo `uninstall(Package p)` per cancellare un pacchetto se questo è installato. È possibile definire `PackageManager2` come sottoclasse di `PackageManager` in accordo con il principio di sostituzione?

Soluzione

Non è possibile, in quanto l'aggiunta del metodo violerebbe un invariante della classe, ovvero il fatto che i pacchetti non possono essere rimossi.

Esercizio 2

Si consideri la seguente classe Java:

```
public class Matrix {
  private double[][] data;
  private int size;

  public Matrix(int size) {
    this.size = size;
    data = new double[size][size];
  }

  public synchronized double get(int row, int column) {
    return data[row][column];
  }

  public synchronized void set(int row, int column, double value) {
    data[row][column] = value;
  }

  public synchronized void add(Matrix m) {
    for(int r=0; r<size; r++) {
      for(int c=0; c<size; c++) {
        data[r][c] += m.data[r][c];
      }
    }
  }

  public synchronized void clear() {
    for(int r=0; r<size; r++) {
      for(int c=0; c<size; c++) {
        data[r][c] = 0;
      }
    }
  }
}
```

Domanda a)

La classe è correttamente sincronizzata? In caso negativo si illustri come modificare la classe per risolvere il problema di sincronizzazione identificato.

Soluzione

Il metodo `add` accede a `m.data` senza sincronizzarsi su `m`. La seguente versione modificata del metodo aggira il problema senza rischiare di cadere in una situazione opposta, di potenziale deadlock:

```
public void add(Matrix m) {
  for(int r=0; r<size; r++) {
    for(int c=0; c<size; c++) {
      set(r,c,get(r,c)+m.get(r,c));
    }
  }
}
```

Domanda b)

Si aggiunga un metodo `public void asyncPrint()` che stampa a video il contenuto della matrice **in un thread separato rispetto al chiamante**. Ovvero, il chiamante dovrà tornare subito mentre un thread separato esegue la vera e propria stampa.

Soluzione

Aggiungiamo un metodo privato `print` che esegue la stampa (deve essere sincronizzato poichè accede allo stato della matrice), mentre il metodo `asyncPrint` (non è necessario che sia sincronizzato) si occuperà di creare ed avviare il thread separato che esegue `print`:

```
public void asyncPrint() {
  new Thread( () -> print() ).start();
  /* Oppure:
  new Thread() {
    public void run() {
      print();
    }
  }.start();
  */
}

private synchronized void print() {
  for(int r=0; r<size; r++) {
    for(int c=0; c<size; c++) {
      System.out.printf("%10.1f ", data[r][c]);
    }
    System.out.println("");
  }
}
```

Domanda c)

Si supponga che il metodo `add` non sia presente e si modifichi il metodo `set` in maniera che il chiamante venga sospeso in attesa che il valore in posizione `row,column` diventi zero prima di impostarlo al valore `value`.

Si indichi se altri metodi devono essere modificati e come (si ricorda che il metodo `add` è stato eliminato).

Soluzione

Il metodo `set` deve essere modificato come segue:

```
private synchronized void set(int row, int column, double value) throws InterruptedException {
  while(data[row][column]!=0) wait();
  data[row][column] = value;
}
```

Inoltre bisogna aggiungere l'istruzione `notifyAll()` al metodo `clear`.

Esercizio 3

Si consideri il seguente programma Java.

```
public abstract class Vector {
   public abstract double norm();
}

public class PlanarVector extends Vector {
    protected final double x;
    protected final double y;
    public PlanarVector(double x, double y) {this.x=x; this.y=y;}
    public double norm(){ return Math.sqrt(x*x+y*y); }
    public PlanarVector sum(PlanarVector v) {return new PlanarVector(x+v.x, y+v.y);}
    public String toString() {return "x=" + x + " y=" + y;}
}

public class SpaceVector extends PlanarVector {
    protected final double z;
    public SpaceVector(double x, double y,double z) {super(x,y); this.z=z;}
    public double norm(){ return Math.sqrt(x*x+y*y+z*z); }
    public SpaceVector sum(SpaceVector v) {return new SpaceVector(x+v.x, y+v.y, z+v.z);
}
    public String toString() {return super.toString() + " z=" + z;}
}
```

e la seguente classe di prova (situata nello stesso package).

```
1  public class TestVector {
2     public static void main (String[] args) {
3         Vector v0;
4         PlanarVector v1, v2;
5         SpaceVector v3, v4;
6         v1 = new PlanarVector (3,4);
7         v0 = v1;
8         v2 = new SpaceVector(2,3,6);
9         v3 = new SpaceVector(2,1,0);
10        v4 = v3;
11        System.out.println(v0.sum(v1));
12        System.out.println(v1.sum(v2));
13        System.out.println(v2.sum(v1));
14        System.out.println(v2.sum(v3));
15        System.out.println(v3.sum(v2));
16        System.out.println(v3.sum(v4));
17        System.out.println(v1.norm());
18        System.out.println(v2.norm());
19    }
20 }
```

Domanda a)

Si elenchino i numeri di riga corrispondenti alle eventuali istruzioni che generano errori di compilazione, giustificando la risposta.

Dopo avere eliminato le istruzioni corrispondenti, si scriva il risultato dell'esecuzione del programma (riportando esplicitamente i numeri di riga originali). Qualora un risultato non sia intero, lo si rappresenti simbolicamente, senza calcolarne il valore (es. "radice di 2").

Soluzione

L'unica istruzione da eliminare è quella alla riga 11, in quanto il tipo statico di v0 (ossia Vector) non ha il metodo `sum`.

Il risultato dell'esecuzione è il seguente:

```
12  X=5.0  y=7.0
13  x=5.0  y=7.0
14  x=4.0  y=4.0
15  x=4.0  y=4.0
16  x=4.0  y=2.0  z=0.0
17  5.0
18  7.0
```

Esercizio 4

Si consideri il seguente metodo statico Java.

```
1  static int test(int x) {
2    int y = x%3;
3    System.out.println(y);
4    while (x>0 && y<3) {
5      if (y==1)
6        return y;
7      y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Domanda a)

Copertura delle istruzioni.

Soluzione

Servono almeno due casi di test, in quanto vi sono due return. Ad esempio, `x=2` (o `x=5`) esegue il cammino `1,2,3,4,5,7,4,5,6`. Il caso `x=0` copre anche la riga 9 (ignoriamo le righe 8 e 10 che sono solo dei terminatori sintattici).

Domanda b)

Copertura delle decisioni (branch).

Soluzione

I due casi precedenti vanno bene.

Domanda c)

Copertura delle condizioni e decisioni.

Soluzione

La copertura totale delle condizioni è impossibile, perché il resto della divisione per 3 è sempre minore di 3. Usando i due casi precedenti è quindi possibile coprire solo 2 condizioni su 3 (66%).

Domanda d)

In base eventualmente anche ai casi di test definiti, individuare la presenza di un possibile errore di programmazione.

Soluzione

Il programma entra in loop infinito quando x%3 è zero. Quindi ad esempio per x=3.